Module 6 — Spring Framework (IoC, DI, Beans)

> Highest priority. This is the core of every Spring interview. If you can explain IoC, DI, and the bean lifecycle crisply, you clear most Spring rounds.

Node.js bridge: In Express you `require()` and `new` your dependencies yourself. Spring *inverts* that: a **container** creates and wires your objects for you (like a built-in, type-safe DI framework — think NestJS, which was inspired by Spring/Angular).

6.1 Spring Architecture & IoC (Inversion of Control)

1. Why Interviewers Ask This

IoC/DI is the whole point of Spring. “What is IoC?” and “What is the difference between IoC and DI?” are near-guaranteed openers.

2. Core Concept

- **IoC (Inversion of Control)** — you don’t create/wire dependencies; the **container** does. Control of object lifecycle is inverted from your code to the framework.
- **DI (Dependency Injection)** — the *technique* IoC uses: dependencies are **injected** (constructor/setter/field) rather than looked up or `new`ed.
- **IoC Container** — `ApplicationContext` (`BeanFactory` is the lower-level base) creates, configures, wires, and manages **beans**.

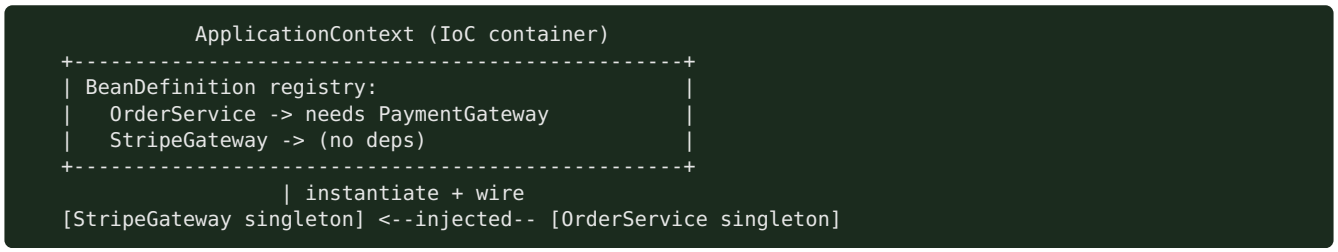
> IoC is the principle; DI is the implementation of that principle. This exact distinction is a frequent follow-up.

3. Internal Working — how the container boots

```
1. Read metadata: @ComponentScan finds @Component/@Service/... ; @Configuration @Bean methods
2. Build BeanDefinitions (class, scope, dependencies, init/destroy) in a registry
3. Instantiate singletons (eagerly by default)
4. Populate dependencies (resolve @Autowired) -> dependency graph
5. Run BeanPostProcessors (this is where AOP proxies, @Transactional, @Async wrap beans)
6. Call init callbacks (@PostConstruct / InitializingBean / init-method)
7. Container ready; beans served on demand
```

BeanDefinitions are metadata; the container uses them to build the object graph, detecting cycles and ordering creation.

4. Memory Diagram



5. Real Production Example

Your `OrderService` needs a `PaymentGateway`. You never write `new StripeGateway()`. You declare the dependency; Spring injects the right implementation, and in tests injects a mock — that’s the productivity and testability win of IoC.

6. Most Asked Questions

- What is IoC? Difference between IoC and DI? *(principle vs technique)*
- What is the IoC container / `ApplicationContext` vs `BeanFactory`? *(ApplicationContext = superset: events, i18n, AOP, eager singletons)*
- How does Spring know what to inject? *(by type, then by name/qualifier)*
- Benefits of DI? *(loose coupling, testability, single responsibility)*

7. Traps

- Saying IoC and DI are the same thing.
- Not knowing `ApplicationContext` extends `BeanFactory`.
- Thinking Spring uses reflection “magic” with no metadata (it builds BeanDefinitions).

8. Best Answer

> “IoC means the framework, not my code, controls object creation and wiring. DI is how it does that — injecting collaborators instead of me `new`-ing them. The `ApplicationContext` reads bean metadata, builds a dependency graph, instantiates singletons, injects dependencies, wraps them with proxies via BeanPostProcessors for cross-cutting concerns, and runs init callbacks. The payoff is loose coupling and trivially mockable tests.”

9. Coding Example

```
@Service
class OrderService {
    private final PaymentGateway gateway;          // depends on abstraction
    OrderService(PaymentGateway gateway) {         // constructor injection (preferred)
        this.gateway = gateway;
    }
}
@Component
class StripeGateway implements PaymentGateway { /* ... */ }
// Spring wires StripeGateway into OrderService automatically.
```

10. Follow-ups

- What if two beans implement `PaymentGateway`? *(`@Primary` or `@Qualifier`)*
- How do you inject a mock in a unit test? *(constructor injection → pass the mock directly, no Spring needed)*

11 & 12. Summary + Cheat

IoC = container controls; DI = injection technique. ApplicationContext ⊃ BeanFactory.

6.2 Dependency Injection — Constructor vs Field vs Setter

Types

- **Constructor injection (preferred)** — dependencies final, immutable, mandatory; enables testing without Spring; fails fast if missing; detects circular deps at startup.
- **Setter injection** — optional/changeable dependencies.
- **Field injection (`@Autowired` on a field)** — concise but **discouraged**: can’t make fields final, hides dependencies, hard to unit test without reflection, allows circular deps to slip through.

Best Answer

> “I use constructor injection: it makes dependencies explicit and final, guarantees a fully initialized object, works in plain unit tests without the Spring context, and surfaces circular dependencies at startup. Field injection is convenient but untestable and hides the contract, so I avoid it.”

```
// PREFERRED - constructor injection (no @Autowired needed if single constructor)
@Service
class UserService {
    private final UserRepository repo;
    private final EmailClient email;
    UserService(UserRepository repo, EmailClient email) { this.repo = repo; this.email = email; }
}
```

Circular dependency

`A → B → A` via constructor injection **fails at startup** (`BeanCurrentlyInCreationException`). Fixes: redesign (best), `@Lazy` on one, or setter injection. Interviewers love this.

6.3 Beans, Scopes, Lifecycle

What is a Bean?

Any object instantiated, assembled, and managed by the Spring IoC container. Defined via stereotype annotations (`@Component` and friends) or `@Bean` methods in `@Configuration`.

Bean Scopes

Scope	Meaning
singleton (default)	one shared instance per container
prototype	new instance every injection/`getBean`
request	one per HTTP request (web)
session	one per HTTP session (web)
application	one per ServletContext

Trap: singleton beans are **shared** — keep them **stateless**. A prototype injected into a singleton is created **once** (at wiring) unless you use `ObjectProvider`/`@Lookup`/scoped proxy.

Bean Lifecycle (guaranteed question)

```
Instantiate -> Populate properties (DI) -> *Aware callbacks (BeanNameAware...) ->
BeanPostProcessor.postProcessBeforeInitialization ->
@PostConstruct -> InitializingBean.afterPropertiesSet() -> custom init-method ->
BeanPostProcessor.postProcessAfterInitialization (AOP proxy created here) ->
[BEAN READY / IN USE] ->
@PreDestroy -> DisposableBean.destroy() -> custom destroy-method
```

- `@PostConstruct` — run init logic after DI (cache warmup, validation).
- `@PreDestroy` — cleanup (only reliably called for singletons, not prototypes).
- **BeanPostProcessor** is where Spring wraps beans in **AOP proxies** — this is how `@Transactional`, `@Async`, `@Cacheable`, and security work.

Memory Diagram

```
new Bean() -> inject deps -> @PostConstruct -> [proxy? wrap via BPP] -> READY
                                                                        |
                                        (@Transactional method call goes through proxy)
```

6.4 Stereotype & Wiring Annotations

Annotation	Meaning
`@Component`	generic Spring-managed bean
`@Service`	business logic (semantic `@Component`)
`@Repository`	data access; also **translates persistence exceptions** to `DataAccessException`
`@Controller`	web MVC controller (returns views)
`@RestController`	`@Controller` + `@ResponseBody` (returns JSON/body)
`@Configuration`	class of `@Bean` definitions
`@Bean`	method producing a container-managed bean (for 3rd-party classes you can't annotate)
`@ComponentScan`	discover stereotypes in packages
`@Autowired`	inject by type
`@Qualifier("name")`	disambiguate among multiple candidates
`@Primary`	default candidate when multiple exist

`@Component` vs `@Bean`

- `@Component` — class-level, auto-detected by scanning; for **your** classes.
- `@Bean` — method-level in `@Configuration`; for **third-party** classes or when you need construction logic.

`@Qualifier` vs `@Primary`

Multiple `PaymentGateway` beans → ambiguity. `@Primary` picks a default globally; `@Qualifier` picks a specific one at the injection point (overrides `@Primary`).

```
@Configuration
class AppConfig {
    @Bean @Primary PaymentGateway stripe() { return new StripeGateway(); }
    @Bean @Qualifier("paypal") PaymentGateway paypal() { return new PaypalGateway(); }
}
@Service
class Checkout {
    Checkout(@Qualifier("paypal") PaymentGateway gw) { /* gets paypal */ }
}
```

`@Configuration` internal detail (nice senior point)

`@Configuration` classes are themselves CGLIB-proxied so that inter-`@Bean` method calls return the **same singleton** rather than a new instance (unlike plain `@Bean` in a `@Component`, "Lite" mode). A frequent deep follow-up.

Module 6 — Top 25 Interview Questions (senior answers)

1. **IoC vs DI?** Principle (container controls) vs technique (inject dependencies).
2. **ApplicationContext vs BeanFactory?** Superset: eager singletons, events, i18n, AOP, auto-BPP.
3. **What is a bean?** Container-managed object built from a BeanDefinition.
4. **Bean scopes?** singleton (default), prototype, request, session, application.
5. **Default scope pitfall?** Singleton shared → must be stateless.
6. **Bean lifecycle?** Instantiate → DI → Aware → BPP-before → @PostConstruct → afterPropertiesSet → init → BPP-after → ready → @PreDestroy → destroy.
7. **@PostConstruct/@PreDestroy?** Init after DI / cleanup before destroy.
8. **What is a BeanPostProcessor?** Hook that wraps beans (AOP proxies) around init.

9. **Constructor vs field vs setter injection?** Immutable/testable vs concise-but-discouraged vs optional.
10. **Why avoid field injection?** No final, hidden deps, hard to test, hides cycles.
11. **Circular dependency handling?** Constructor cycle fails fast; fix by redesign/@Lazy/setter.
12. **@Component vs @Service vs @Repository vs @Controller?** Semantics; @Repository adds exception translation.
13. **@Controller vs @RestController?** Views vs JSON body.
14. **@Component vs @Bean?** Auto-scanned class vs method for third-party/factory.
15. **@Autowired resolution order?** By type → by qualifier/name; @Primary as default.
16. **@Qualifier vs @Primary?** Point-specific vs global default; qualifier wins.
17. **@ComponentScan?** Discovers stereotypes in base packages.
18. **Is @Autowired required?** Optional on a single constructor since Spring 4.3.
19. **Prototype in singleton problem?** Injected once; use ObjectProvider/@Lookup/scoped proxy.
20. **How is @Transactional applied?** Via AOP proxy created by a BeanPostProcessor.
21. **@Configuration CGLIB proxy?** Ensures @Bean calls return the same singleton.
22. **Lazy vs eager beans?** Singletons eager by default; `@Lazy` defers creation.
23. **How to define a bean for a 3rd-party class?** `@Bean` in `@Configuration`.
24. **Spring vs Spring Boot?** Framework (DI/MVC) vs opinionated auto-config on top.
25. **How to unit test a service?** Constructor-inject mocks; no context needed.

Module 6 — Top Coding Questions

- Wire two implementations and select with `@Qualifier` / `@Primary`.
- Demonstrate the bean lifecycle with `@PostConstruct` / `@PreDestroy` logging.
- Reproduce a circular dependency and fix it.
- Register a third-party client (e.g., `RestTemplate` / `WebClient`) as a `@Bean`.

Module 6 — Common Follow-ups

- “What actually happens at `context.getBean(OrderService.class)`?”
- “Why must singletons be stateless?”
- “How does `@Transactional` work under the hood?” (proxy + BeanPostProcessor.)

Module 6 — One-Page Cheat Sheet

```
IoC = container controls creation/wiring. DI = injection technique. ApplicationContext ⊃ BeanFactory
Lifecycle: instantiate->DI->Aware->BPP.before->@PostConstruct->afterPropertiesSet->init->BPP.after-
>READY->@PreDestroy->destroy
BeanPostProcessor wraps AOP proxies (@Transactional/@Async/@Cacheable)
Scopes: singleton(default, stateless!) prototype request session application
Injection: constructor(preferred, final, testable) > setter(optional) > field(avoid)
Circular ctor dep -> fails fast -> redesign/@Lazy/setter
@Component(class,scan) vs @Bean(method,3rd-party). @Repository=exception translation
@Autowired by type; @Qualifier(point) beats @Primary(default)
@Configuration is CGLIB-proxied -> @Bean calls share singleton
```

Module 6 — Mock Interview (answer, then continue)

1. “Explain IoC vs DI to a Node developer, with a concrete example.”
2. “Walk through the full bean lifecycle and tell me exactly where `@Transactional` gets applied.”

3. “You have two `PaymentGateway` beans; injection fails. Two ways to fix it and which you’d choose.”
4. “Why is constructor injection better than field injection? Convince me.”
5. “A prototype-scoped bean injected into a singleton isn’t behaving as expected — why, and how do you fix it?”

Continue to Module 7 when ready.